

CS514L-Artificial Intelligence

# **Assignment 3**

## **Game Playing**

**Team Members**

CS25S002 - J ARJUN ANNAMALAI

CS24M106 - LOGESH .V

# Minimax Algorithm

The Minimax algorithm is a decision-making algorithm for determining the best move in a two-player game like chess. It works by:

1. Recursively exploring all possible moves up to a certain depth
2. Evaluating positions at the leaves of the search tree
3. Propagating values back up the tree, with the maximizing player choosing maximum values and the minimizing player choosing minimum values

## Algorithm Characteristics:

- **Complete:** Guarantees finding the optimal solution within the search depth
- **Exponential Complexity:**  $O(b^d)$  where  $b$  is branching factor ( $\sim 35$  in chess) and  $d$  is depth
- **Memory Usage:**  $O(bd)$  for storing the recursion stack
- **Decision Quality:** Directly proportional to search depth
- **Predictable:** Always chooses the same move in identical positions



# $\alpha\beta$ -pruning Algorithm

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm that significantly reduces the number of nodes that need to be evaluated in the search tree. It maintains two values, alpha and beta, which represent the minimum score that the maximizing player is assured of and the maximum score that the minimizing player is assured of, respectively. If at any point during the search, alpha becomes greater than or equal to beta, the remainder of that branch can be pruned (ignored).

## Algorithm Characteristics:

- **Same Completeness:** Produces identical decisions to Minimax
- **Improved Efficiency:** Best case:  $O(b^{(d/2)})$  - equivalent to doubling search depth
- **Variable Performance:** Efficiency depends on move ordering
- **Earlier Cutoffs:** Better with good move ordering heuristics
- **Same Memory Requirements:**  $O(bd)$  - no additional memory overhead compared to Minimax



# Evaluation function

## Material-based evaluation:

Pawn = 1 point, Knight = 3 points, Bishop = 3 points, Rook = 5 points, Queen = 9 points, King = 0 points

## Working Mechanism:

- Iterates through all 64 squares on the board
- Identifies pieces and their types
- Adds value for White pieces and subtracts for Black pieces
- Returns a score representing White's advantage/disadvantage

## Strengths and Limitations

**Strengths:** Simple, computationally efficient, based on established chess theory

**Limitations:** Doesn't consider positional factors, king safety, or tactical patterns

# Performance Metrics

## Alpha-Beta Pruning (Depth 3)

- Achieves checkmate in 63 moves
- Makes more aggressive attacking moves
- Prioritizes controlling the center of the board
- Shows evidence of multi-move tactics

## Minimax (Depth 2)

- Reaches stalemate after 179 moves
- Makes more defensive and reactive moves
- Less structured pawn formation
- Primarily focused on immediate material gains

| Metric                   | Alpha-Beta (d=3) | Minimax (d=2)    | Improvement                        |
|--------------------------|------------------|------------------|------------------------------------|
| Moves to completion      | 63               | 179              | 64.8% fewer moves                  |
| Runtime (seconds)        | 31.95            | 45.84            | 30.3% faster                       |
| Result quality           | Checkmate (win)  | Stalemate (draw) | Superior outcome                   |
| Avg. time per move (sec) | 0.51             | 0.26             | Deeper search despite only 2x time |